



The Problem

Every week, a misconfigured S3 bucket or open SSH port goes undetected in a Terraform commit. By the time it reaches production, the blast radius is a compliance incident or a breach.

Gap	Reality
Who writes IaC	Infrastructure engineers
Who audits it	Security engineers
When it's reviewed	Too late — after the PR merges
Current tooling	Manual, inconsistent, slow

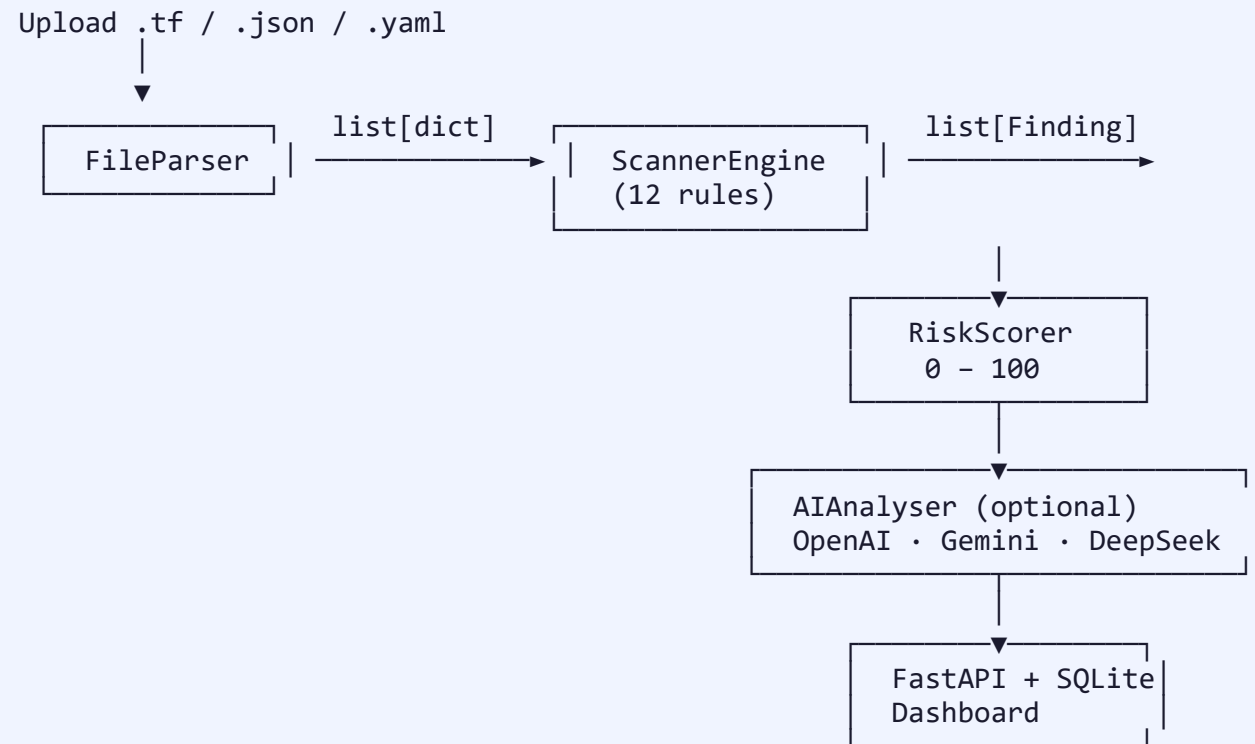
WK's ask: A scanner that catches misconfigurations *before* they ship — with a visual risk score and actionable remediation.

Solution in 30 Seconds

Upload main.tf → Risk Score: 100 → "Your RDS instance is publicly accessible. Set publicly_accessible = false immediately."

	Capability
	Upload <code>.tf</code> , <code>.json</code> , <code>.yaml</code> — Terraform, CloudFormation, Pulumi
	0–100 risk score returned in under 5 seconds
	Every finding includes the exact fix — not just a flag
	Visual dashboard: gauge, severity charts, scan history
	Optional AI executive summary (OpenAI / Gemini / DeepSeek)

Architecture



12 Security Rules

Severity	Rule	Trigger
● CRITICAL	S3_PUBLIC_ACL	acl = "public-read"
● CRITICAL	SSH_OPEN_TO_WORLD	Port 22 → 0.0.0.0/0
● CRITICAL	RDP_OPEN_TO_WORLD	Port 3389 → 0.0.0.0/0
● CRITICAL	WILDCARD_IAM_ACTION	IAM action "*"
● HIGH	UNENCRYPTED_EBS	encrypted = false
● HIGH	UNENCRYPTED_RDS	storage_encrypted = false
● HIGH	PUBLIC_RDS	publicly_accessible = true
● HIGH	HARDCODED_SECRET	password / token / api_key in config
● MEDIUM	S3_VERSIONING_DISABLED	No versioning block
● MEDIUM	CLOUDTRAIL_DISABLED	enable_logging = false
● MEDIUM	UNRESTRICTED_EGRESS	All-port egress → 0.0.0.0/0
● MEDIUM	MISSING_REQUIRED_TAGS	No Name or Environment tag

Formula: `risk_score = min(CRITICAL×40 + HIGH×20 + MEDIUM×5, 100)`

Vibe Coding Workflow

"You are the architect; the AI is the engineer." — WK Challenge Brief

This is exactly how it was built — every turn documented in [prompts.md](#)

Turn	Phase	Module	Gate
01	ARCHITECT	VISION + ARCHITECTURE + ADR + SPEC	Approved before any code
02	SCAFFOLD	Health endpoint only	ruff ✓ mypy ✓ bandit ✓
03–05	RED→GREEN→REFACTOR	FileParser	hcl2 bug caught + fixed
06–07	RED→GREEN	ScannerEngine	Pydantic Finding model
08–09	RED→GREEN	RiskScorer	5-line implementation
10–11	RED→GREEN	API Routes	Full Pydantic schema layer
12	DEMO	Dashboard	Bootstrap 5 + Chart.js
13	SMOKE	20 edge cases	Versioning block bug caught

Deviation note logged when discipline slipped — visible in [prompts.md](#).

Pydantic v2 as Governance

The challenge asked for compliance tooling. The code demonstrates compliance discipline.

```
class Finding(BaseModel):
    rule_id: str
    severity: Literal["CRITICAL", "HIGH", "MEDIUM"] # enum enforced
    title: str
    description: str
    remediation: str # never empty
    resource_name: str # full provenance
    resource_type: str
    model_config = {"frozen": True} # immutable

class ScanResponse(BaseModel):
    risk_score: int | None = Field(None, ge=0, le=100) # range constrained
    findings: list[FindingResponse] # typed, not raw dicts
```

A finding with severity **"BANANA"** raises **ValidationError**. It never reaches the database.

Quality Gates — Every Commit

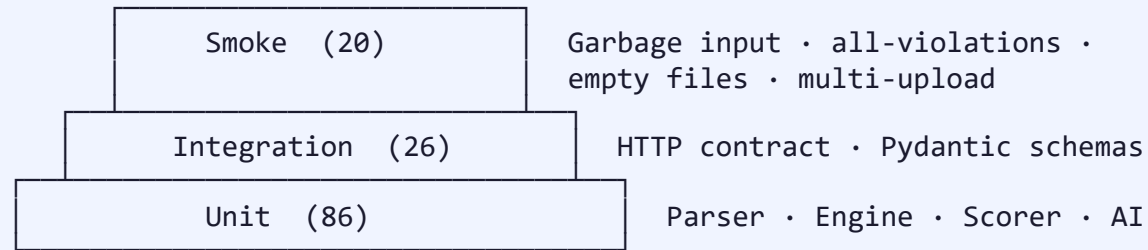
<code>ruff check app/ tests/</code>	→ linting + import order
<code>mypy app/ --strict</code>	→ full type safety, no Any leakage
<code>bandit -r app/</code>	→ security scan
<code>pip-audit</code>	→ dependency vulnerability check
<code>pytest --cov-fail-under=85</code>	→ 132 tests · 87% coverage floor

All 5 gates run automatically on every push via GitHub Actions CI



The badge at the top of README.md is live. If CI is green, the code is clean.

Test Pyramid



What smoke tests caught that unit tests missed:

Bug	How It Was Found
hcl2 returns type names with quotes on Linux	Score was 30 instead of 100 on CI
Versioning block parsed as list, not dict	Clean bucket scored 5 instead of 0

These are **production-grade bugs** caught before submission.

AI Executive Summary — Live

With any AI key set, every scan returns:

```
{  
  "risk_score": 100,  
  "summary": "Overall Risk Level: Critical. The S3 bucket data_lake  
              poses a critical risk due to a public ACL. SSH port 22  
              is exposed to all IPs, enabling brute-force attacks.  
              The RDS prod_db is unencrypted and publicly accessible.  
  
              Recommended first action: Remove the public ACL from  
              data_lake and restrict SSH access immediately.",  
  "findings": [...]  
}
```

Three providers — one SDK, zero vendor lock-in:

`OPENAI_API_KEY` → gpt-4o-mini · `GEMINI_API_KEY` → gemini-2.0-flash · `DEEPSEEK_API_KEY` → deepseek-chat

What This Means for Wolters Kluwer

WK platform teams ship IaC daily. Every Terraform PR is a potential compliance incident.

Direct CI/CD integration:

```
curl -F "file=@main.tf" https://guardrail.internal/api/scan | jq '.risk_score'  
# 0    → merge allowed  
# 100 → block PR + notify security team
```

Extending to WK compliance needs — zero changes outside `rules.py`:

New Rule	WK Use Case
<code>MISSING_COST_CENTER_TAG</code>	FinOps — enforce cost attribution
<code>NON_APPROVED_REGION</code>	Data residency compliance
<code>CIS_BENCHMARK_*</code>	CIS AWS Foundations alignment

Submission Checklist

	Deliverable	Status
✓	Tagle.ai Profile	The Navigator · AI Operator · AI Ready · 73/100
✓	Public GitHub Repository	github.com/Sebuliba-Adrian/wk-security-guardrail-auditor
✓	<code>prompts.md</code> audit log	13 turns · deviation note · bugs caught
✓	Presentation Deck	This deck
✓	Cloud decommissioned	SQLite only — no cloud resources
✓	CI passing	ruff · mypy · bandit · pip-audit · 132 tests
✓	AI summary	OpenAI / Gemini / DeepSeek

